Microservices Communication and Integration Standards

Document ID: ENG-003

Version: 1.0

Department: Engineering

Team: Backend Engineering

Effective Date: March 15, 2026

Last Updated: October 25, 2026

1. Purpose

This document establishes communication and integration standards for microservices developed at Meridian Technologies.

The Backend Engineering team maintains service interfaces, API contracts and asynchronous communication mechanisms.

The organization uses microservices to separate business capabilities and support independent development and deployment.

2. Architecture Overview

The platform contains the following major services and infrastructure components:

Component	Responsibility
API Gateway	Routes client requests
Authentication Service	Validates credentials and tokens
User Service	Manages employee

	accounts
Notification Service	Handles notifications
Transaction Service	Processes business transactions
Document Service	Manages document metadata
Kafka	Asynchronous event communication
PostgreSQL	Persistent relational storage
Redis	Caching and temporary data

Each service must maintain a clearly defined responsibility.

Services must not directly modify another service's internal database tables without an approved integration design.

3. REST API Communication

Synchronous communication primarily uses REST APIs with JSON request and response formats.

Each endpoint must define its HTTP method, request schema, response schema, authentication requirements and error handling.

Example endpoints:

- `GET /internal/users/{user_id}`
- `POST /internal/notifications`
- `GET /internal/documents/{document_id}`

Standard internal API requests should have a three-second timeout.

Authentication validation requests should have a two-second timeout.

Longer timeouts require documented justification.

4. Asynchronous Communication

Kafka is used for asynchronous communication between services.

Typical events include:

- User profile updates.
- Transaction creation.
- Document ingestion completion.
- Notification requests.
- Background processing events.

Events must contain an event identifier, event type, timestamp, source service and correlation ID.

Example:

```
{
  "event_id": "EVT-2026-00421",
  "event_type": "DOCUMENT_INGESTED",
  "service": "document-service",
  "correlation_id": "CORR-10021",
  "status": "completed"
}
```

Sensitive information must not be unnecessarily included in event payloads.

5. Message Reliability

Kafka consumers must support idempotent processing to prevent duplicate business operations.

The standard retry schedule is:

Attempt	Delay
First	5 seconds
Second	30 seconds
Third	120 seconds

Messages that fail after three retries must be routed to a dead-letter queue.

The responsible service team must investigate failures before replaying affected messages.

6. API Versioning

Internal APIs must use explicit versioning when consumed by multiple services.

Examples:

- `/api/v1/users`
- `/api/v2/users`

Breaking changes require an approved migration plan.

The standard migration notice period is 30 calendar days unless an urgent security issue requires earlier action.

7. Database and Caching Standards

PostgreSQL remains the authoritative source for persistent relational information.

Redis may cache frequently accessed records to reduce database load.

Cached values must not be treated as the authoritative source for financial or transactional records.

Database schema changes must follow controlled migration procedures.

8. Monitoring and Reliability

Services must expose metrics for request latency, error rates, request throughput, database connections and Kafka consumer lag.

Correlation IDs should be maintained across related service operations.

Services must provide health-check endpoints and structured application logs.

Passwords, authentication tokens and confidential employee information must never appear in logs.

9. Security Requirements

All service communication must follow approved authentication and authorization mechanisms.

Services must validate incoming requests and apply least-privilege access.

Internal network placement alone is not sufficient authorization.

Secrets must be stored using approved secret management mechanisms.

10. Responsibilities

The Backend Engineering Lead owns these integration standards.

Individual service teams maintain their API contracts, event schemas and consumer implementations.

Changes affecting multiple services must be reviewed by the relevant service owners.

11. Related Documents

- ENG-001: Backend Platform Architecture and API Design
- ENG-002: Software Engineering Development and Deployment Guidelines
- ENG-004: Production Deployment and Incident Response
- IT-001: Enterprise Access Control Policy
- IT-005: Security Incident Handling Playbook

12. Version History

Version	Date	Description
1.0	2026-03-15	Initial integration standards

Document Owner: Backend Engineering Lead